

Experiment No.	2
Experiment Title	Email Spam/Ham Classification using Naïve Bayes and KNN
Student Name	Radesh
Register Number	3122247001047
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	August 16, 2026
Github Repository	https://github.com/RadeshL/Machine-Learning-Laboratory/blob/main/MLassign2.ipynb

1 Objective

The objective of this experiment is to build and compare machine learning classifiers for detecting spam email using the Spambase dataset. Specifically, the experiment aims to:

- Build spam classifiers using Naïve Bayes and K-Nearest Neighbors (KNN).
- Compare the performance of Gaussian, Multinomial and Bernoulli Naïve Bayes variants.
- Study the effect of varying the number of neighbors k on KNN performance.
- Compare the KDTree and BallTree algorithms used for nearest-neighbor search.
- Optimize KNN hyperparameters using GridSearchCV and RandomizedSearchCV.
- Evaluate the computational (training/prediction) time and theoretical time complexity of each algorithm.
- Evaluate all models using 5-Fold Cross Validation.

2 Problem Statement

Given a collection of emails represented as numerical feature vectors (word frequencies, character frequencies and capital-letter run-length statistics), the task is to classify each email as **Spam** or **Not Spam (Ham)**. The input to the models is a 57-dimensional feature vector per email, and the output is a binary class label (1 = spam, 0 = not spam). The objective is to train and compare probabilistic (Naïve Bayes) and instance-based (KNN) classifiers on this task and to identify the model configuration that gives the best trade-off between accuracy and computational cost.

3 Dataset Description

Dataset Name	Spambase Dataset
Dataset Source	Kaggle – https://www.kaggle.com/datasets/somesh24/spambase

Number of Samples	4601 (rows before cleaning); 4210 after removing 391 duplicate rows
Number of Features	57 input features (48 word-frequency, 6 character-frequency, 3 capital-run-length features)
Number of Classes	2 (Spam = 1, Not Spam = 0)
Missing Values	None (0 missing values in any column)
Train-Test Split	80% train / 20% test (3368 training samples, 842 testing samples), <code>random_state=42</code>

4 Exploratory Data Analysis

The Spambase dataset consists of 57 continuous numerical features derived from email text (word frequencies, character frequencies, and statistics of consecutive capital letters) and a binary target column `class`. The dataset was checked for missing values, duplicate rows and constant columns before proceeding with model training.

Missing Value Analysis

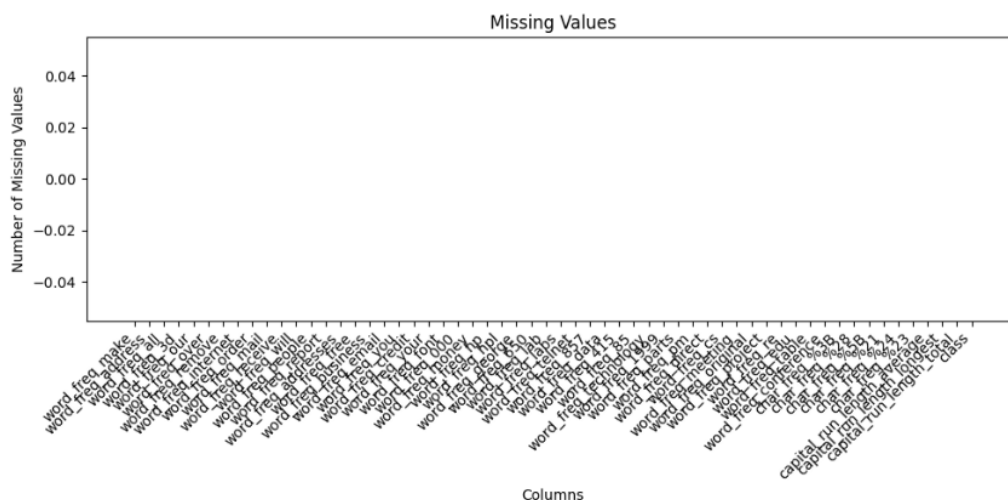


Figure 1: Missing value count per column.

Inference: The bar plot confirms that every feature column has zero missing values, so no imputation was strictly necessary for this dataset. The dataset did, however, contain 391 exact duplicate rows, which were removed prior to analysis to avoid biasing the model with repeated samples. No column exceeded the 50% missing-value threshold and no constant (zero-variance) columns were found. This clean starting point meant preprocessing effort could be focused on scaling and normalization rather than missing-value handling.

Class Distribution

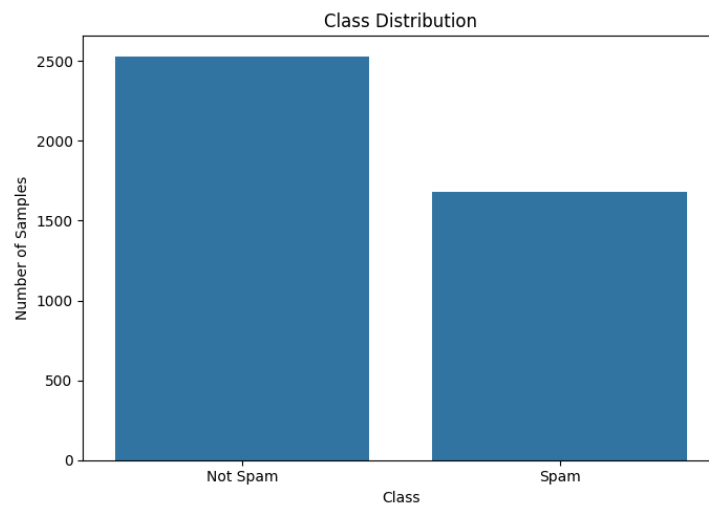


Figure 2: Distribution of Spam vs. Not Spam classes.

Inference: The class distribution plot shows that the dataset is moderately imbalanced, with more "Not Spam" emails than "Spam" emails, though not to an extreme degree. This level of imbalance is mild enough that accuracy remains a reasonably informative metric, but it also justifies reporting precision, recall, F1-score and ROC-AUC alongside accuracy so that performance on the minority (spam) class is not obscured. It further motivated the use of stratified 5-fold cross-validation so that each fold preserves the original class ratio.

Correlation Analysis

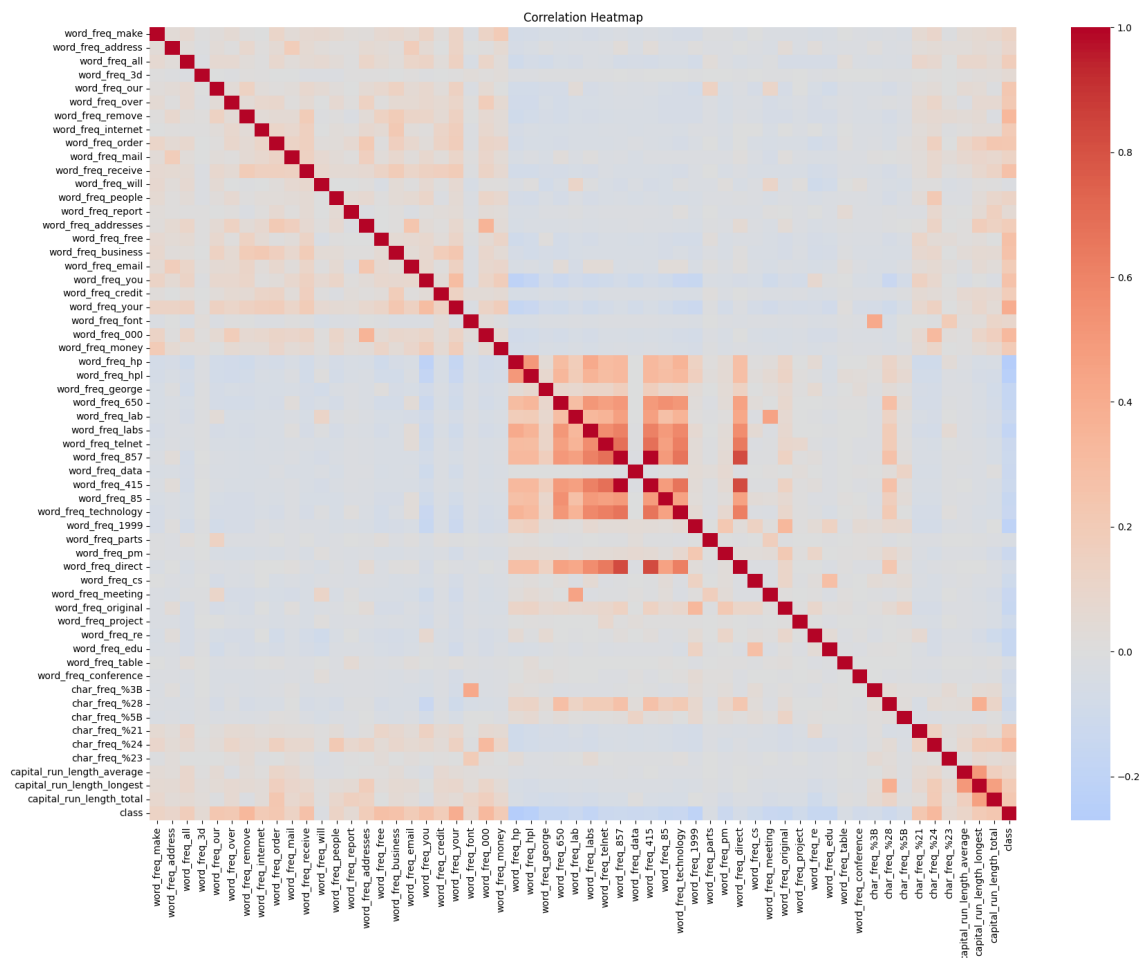
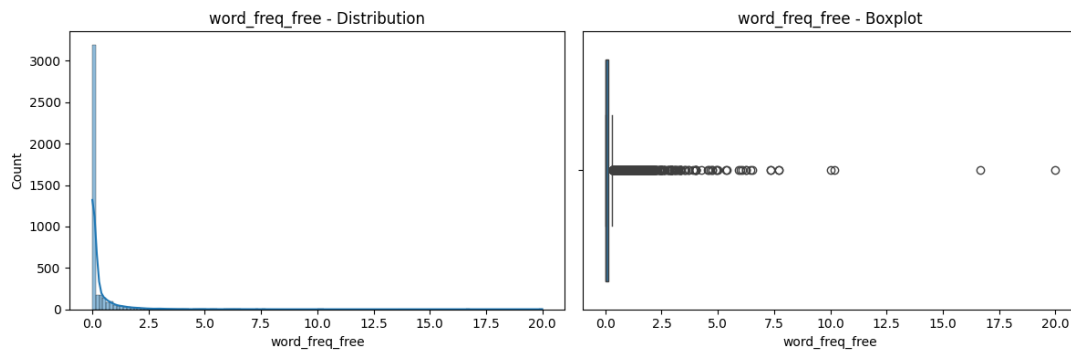
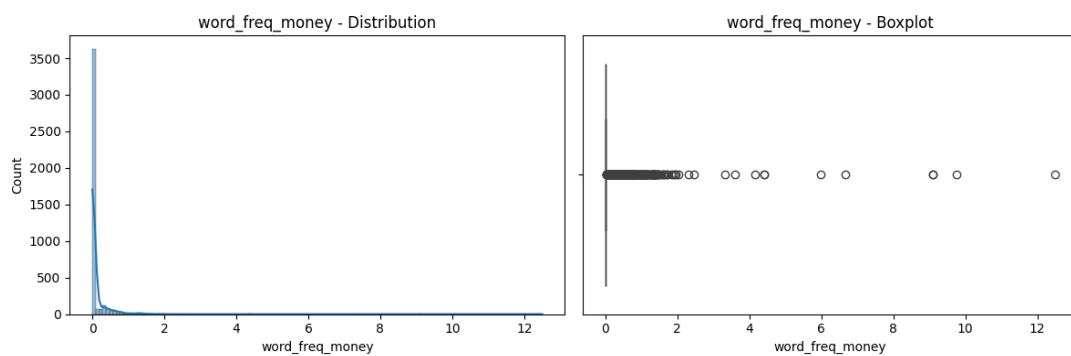
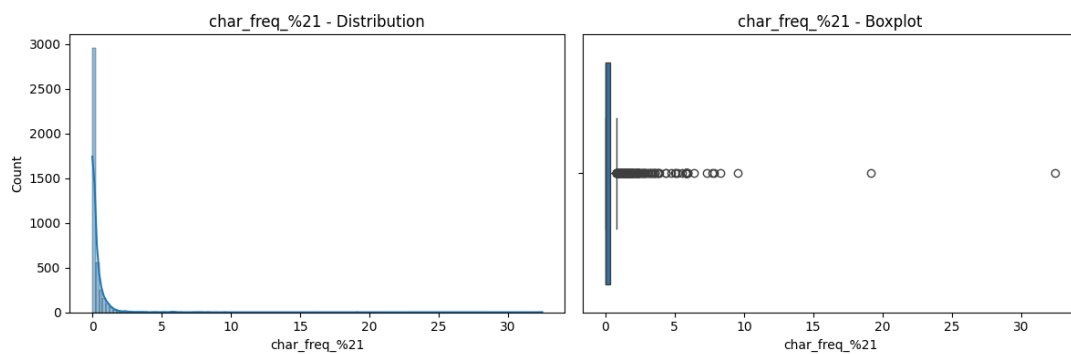


Figure 3: Correlation heatmap of all numerical features.

Inference: The correlation heatmap shows that most word-frequency features are weakly correlated with one another, indicating that they carry largely independent information about the email content. A few clusters of moderately correlated features are visible (for example, among related word-frequency groups and among the capital-run-length statistics), which partially explains why Gaussian and Bernoulli Naïve Bayes – both of which implicitly assume conditional feature independence – still perform competitively despite this simplifying assumption not being perfectly satisfied. This also motivates feature scaling for KNN, since a few features have much larger ranges than others.

Feature Distributions

Representative distribution and boxplot pairs for a few informative features are shown below; the full analysis in the notebook produces this pair of plots for all 57 numerical features.

Figure 4: Distribution and boxplot of `word_freq_free`.Figure 5: Distribution and boxplot of `word_freq_money`.Figure 6: Distribution and boxplot of `char_freq_%21` (frequency of '!').

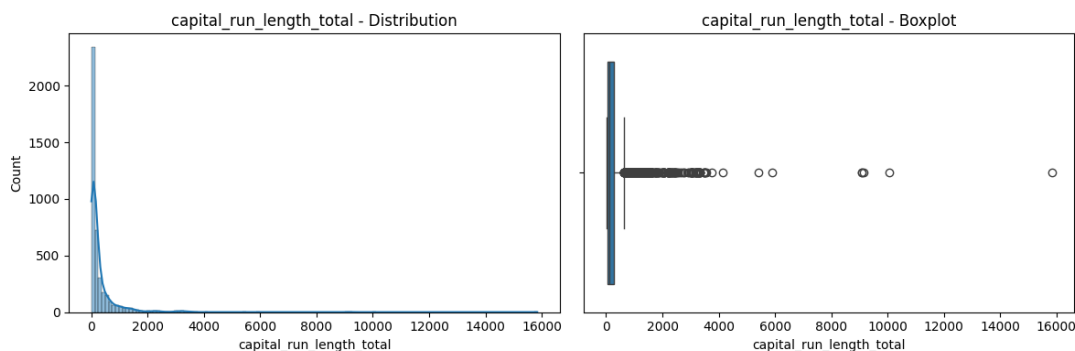


Figure 7: Distribution and boxplot of `capital_run_length_total`.

Inference: All of the sampled features are strongly right-skewed, with the majority of emails having a value close to zero and a long tail of outlying values, as seen clearly in the accompanying boxplots. Words such as “free” and “money” and the frequency of the ‘!’ character occur rarely in ham emails but spike in many spam emails, making them useful discriminative signals. The heavy skew and presence of outliers in features such as `capital_run_length_total` justify the use of standardization (z-score scaling) before distance-based algorithms such as KNN are applied, since KNN is sensitive to feature scale.

5 Data Preprocessing

The following preprocessing steps were applied to the dataset:

- **Missing value handling:** The dataset had no missing values; the pipeline nonetheless included a safeguard that imputes numerical columns with the median and categorical columns with the mode, and drops columns with more than 50% missing values or zero variance.
- **Encoding:** Not required, as all 57 input features and the target were already numeric.
- **Duplicate removal:** 391 duplicate rows were identified and dropped, reducing the effective dataset size.
- **Feature scaling:** For KNN, features were standardized using `StandardScaler` (zero mean, unit variance) fitted on the training set and applied to the test set, since KNN is a distance-based algorithm sensitive to feature magnitude. Naïve Bayes models were trained on the unscaled features, consistent with their probabilistic (not distance-based) formulation.
- **Train-test split:** An 80/20 split was used (`train_test_split, random_state=42`), giving 3368 training samples and 842 testing samples.
- **Feature engineering:** No additional feature engineering was performed; the original 57 Spambase features were used directly.

6 Machine Learning Algorithm

Naïve Bayes

Working principle: Naïve Bayes is a probabilistic classifier that applies Bayes' theorem under the (“naïve”) assumption that features are conditionally independent given the class label.

Mathematical formulation:

$$P(C | X) = \frac{P(X | C) P(C)}{P(X)} \quad (1)$$

where, under the conditional independence assumption, $P(X | C) = \prod_{i=1}^n P(x_i | C)$. The class with the highest posterior probability is predicted. Gaussian NB models $P(x_i | C)$ as a Gaussian distribution, suited to continuous features; Multinomial NB models feature counts/frequencies using a multinomial distribution, typically suited to discrete count data; Bernoulli NB models each feature as a binary indicator (present/absent).

Advantages: Fast to train and predict (linear in the number of features), works well with high-dimensional data, requires relatively little training data, and is naturally probabilistic.

Limitations: The conditional independence assumption is rarely true in practice, which can hurt calibration and, in some cases, accuracy; Gaussian NB additionally assumes features are normally distributed within each class, which does not hold for the strongly skewed Spambase features.

K-Nearest Neighbors (KNN)

Working principle: KNN is a lazy, instance-based learning algorithm. To classify a new sample, it finds the k nearest training samples (by a distance metric such as Euclidean or Manhattan distance) and assigns the majority class among them.

Mathematical formulation: For Euclidean distance between points x and x' in $\mathbb{R}^d$,

$$d(x, x') = \sqrt{\sum_{j=1}^d (x_j - x'_j)^2} \quad (2)$$

and the predicted class is

$$\hat{y} = \text{mode}\{y_i : x_i \in N_k(x)\} \quad (3)$$

where $N_k(x)$ is the set of k nearest neighbors of x in the training set.

Advantages: Simple, non-parametric (makes no assumption about the underlying data distribution), naturally handles multi-class problems, and can model complex, non-linear decision boundaries.

Limitations: Prediction is computationally expensive for large datasets (all distances must be computed at query time for the brute-force variant), it is sensitive to feature scaling and irrelevant features, and it requires storing the entire training set in memory.

7 Experimental Setup

Python Version	3.13
Scikit-Learn Version	1.7.2
IDE	Jupyter Notebook
Operating System	Windows
Hardware	Standard laptop / desktop (CPU only)

8 Hyperparameter Selection

Parameter	Value
<code>n_neighbors</code> search space (GridSearchCV)	{3, 5, 7, 9, 11, 13, 15}
<code>weights</code> search space	{uniform, distance}
<code>metric</code> search space (Grid)	{euclidean, manhattan}
<code>metric</code> search space (Randomized)	{euclidean, manhattan, minkowski}
<code>n_neighbors</code> search space (RandomizedSearchCV)	Uniform integer in [1, 30]
Cross-validation folds	5 (<code>cv=5</code>)
RandomizedSearchCV iterations	20 (<code>n_iter=20</code>)
Best parameters (GridSearchCV)	<code>metric='manhattan', n_neighbors=5, weights</code>
Best CV accuracy (GridSearchCV)	0.9139
Best parameters (RandomizedSearchCV)	<code>metric='manhattan', n_neighbors=6, weights</code>
Best CV accuracy (RandomizedSearchCV)	0.9136
KDTree / BallTree	<code>n_neighbors=5</code> , default (Euclidean) metric

9 Results

Naïve Bayes Comparison

Metric	Gaussian NB	Multinomial NB	Bernoulli NB
Accuracy	0.8468	0.8040	0.8812
Precision	0.7468	0.7870	0.8960
Recall	0.9694	0.7409	0.8162
F1-score	0.8436	0.7633	0.8542
ROC-AUC	0.9550	0.8478	0.9490

KNN and Tuned-KNN Comparison

Model	Accuracy	Precision	Recall	F1	ROC-AUC
KNN (default, $k = 3$)	0.8848	0.8808	0.8440	0.8620	0.5702
GridSearchCV KNN	0.9133	0.9497	0.8412	0.8922	0.7567
RandomizedSearchCV KNN	0.9121	0.9495	0.8384	0.8905	0.8322
KDTree KNN ($k = 5$)	0.8836	0.8943	0.8245	0.8580	0.7028
BallTree KNN ($k = 5$)	0.8836	0.8943	0.8245	0.8580	0.7028

Overall Best Model

Metric	Value (GridSearchCV-tuned KNN)
Accuracy	0.9133
Precision	0.9497
Recall	0.8412
F1-score	0.8922
ROC-AUC	0.7567

5-Fold Cross-Validation

Fold	Bernoulli NB Accuracy	KNN Accuracy
1	0.8902	0.8843
2	0.8902	0.8932
3	0.8828	0.9050
4	0.8722	0.8960
5	0.8975	0.9138
Average	0.8866	0.8985

10 Result Visualization

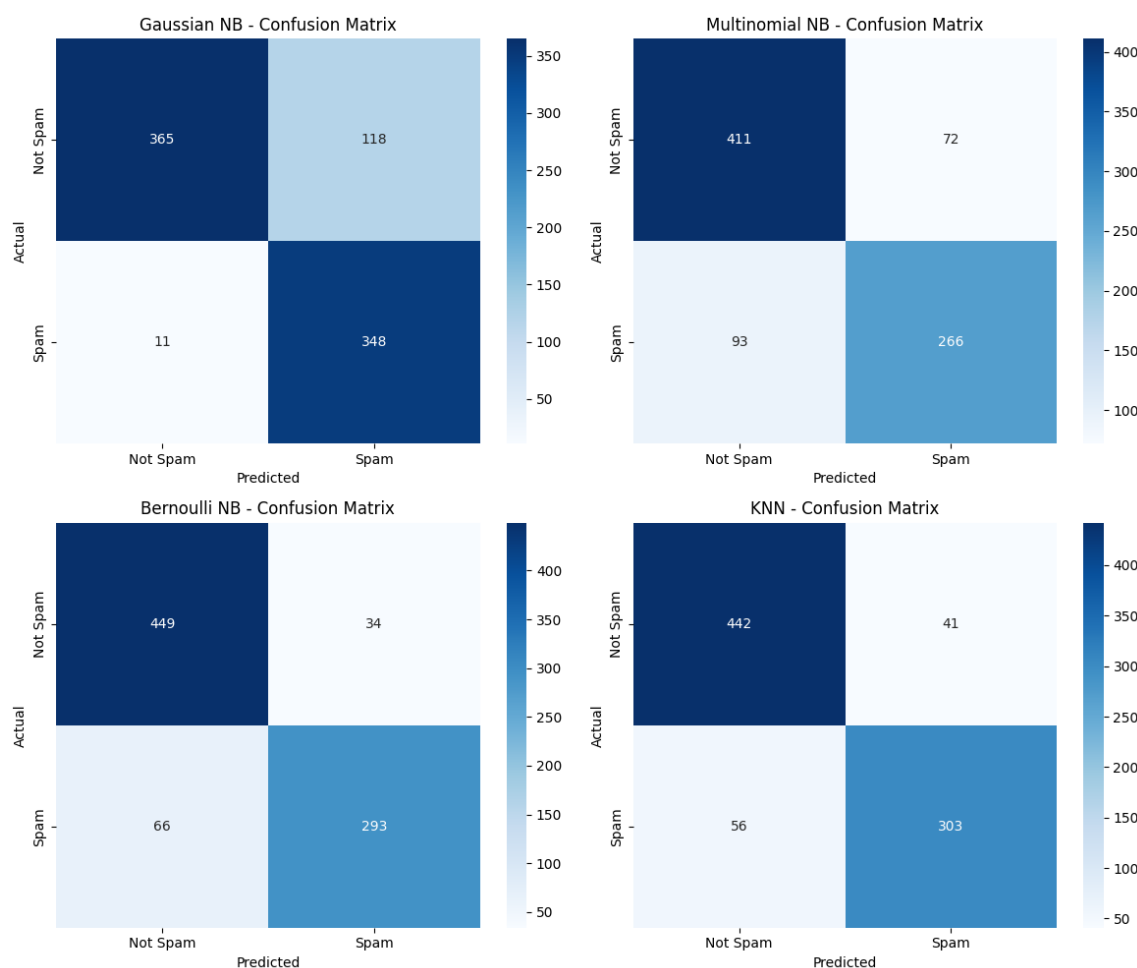


Figure 8: Confusion matrices for Gaussian NB, Multinomial NB, Bernoulli NB and default KNN.

Inference: The confusion matrices show that Gaussian NB produces the fewest false negatives (missed spam), consistent with its high recall of 0.969, but at the cost of a larger number of false positives (ham misclassified as spam). Bernoulli NB and KNN show a more balanced trade-off between false positives and false negatives. Multinomial NB has the highest false-negative count among the four, explaining its comparatively lower recall and F1-score. These matrices highlight that the “best” model depends on whether false positives (blocking legitimate mail) or false negatives (letting spam through) are considered more costly.

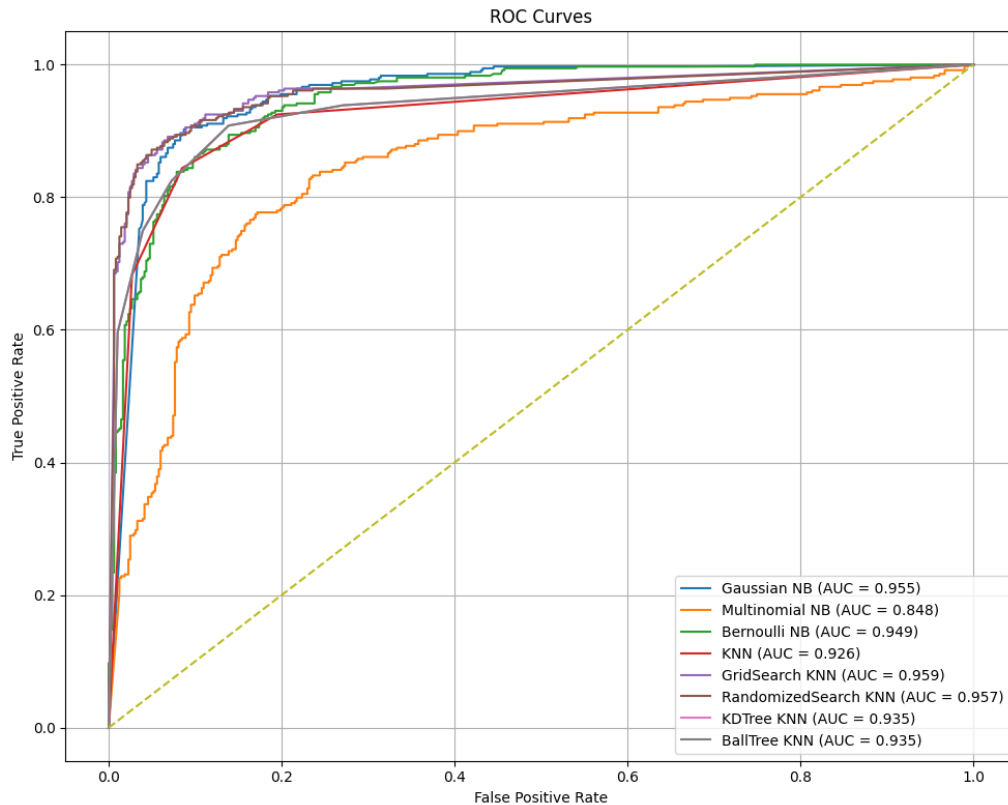


Figure 9: ROC curves for all trained models.

Inference: Gaussian NB and Bernoulli NB achieve the highest ROC-AUC scores (0.955 and 0.949 respectively), indicating strong separability between spam and ham across all classification thresholds. In contrast, the default (unscaled-probability) KNN model shows a notably lower AUC (0.570), even though its accuracy is competitive, illustrating that KNN's `predict_proba` output (based on neighbor vote fractions) is a much coarser probability estimate than the smooth likelihoods produced by Naïve Bayes. Tuning KNN with `GridSearchCV`/`RandomizedSearchCV` substantially improves its ranking ability (AUC rises to 0.757–0.832), showing the value of hyperparameter optimization beyond just accuracy.

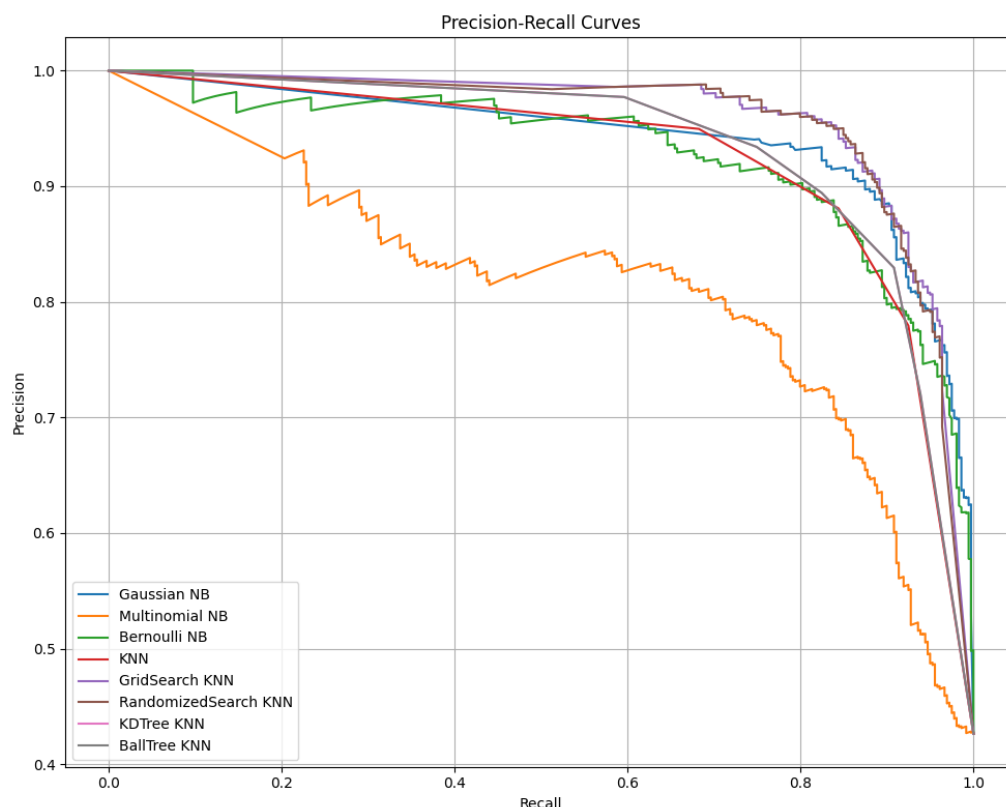


Figure 10: Precision-Recall curves for all trained models.

Inference: The precision-recall curves reinforce the ROC findings: Gaussian and Bernoulli NB maintain high precision across a wide range of recall values, making them attractive when both false positives and false negatives matter. The tuned KNN variants (Grid/RandomizedSearch) achieve high precision at moderate recall, reflecting their conservative “distance-weighted” voting that favors confidently-close neighbors. This view is particularly informative for the spam-detection task, where the positive (spam) class is the minority and precision-recall curves are less optimistic (and more realistic) than ROC curves under class imbalance.

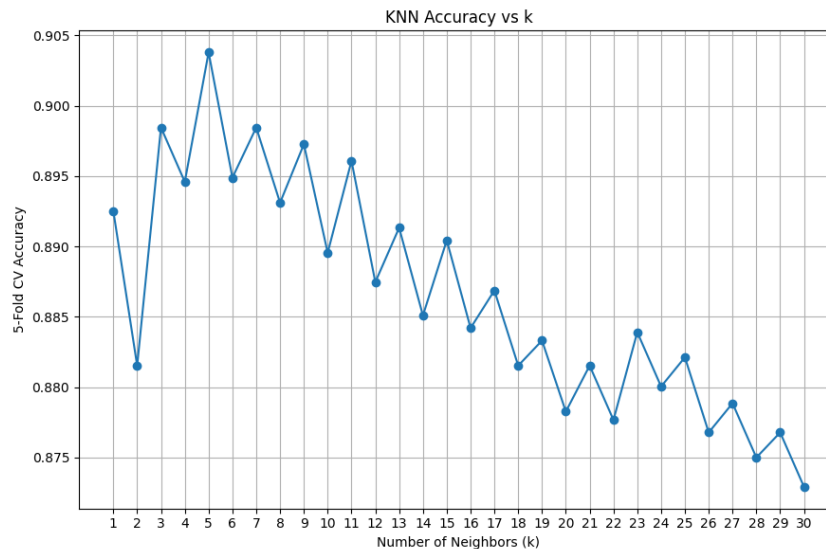


Figure 11: 5-fold cross-validated KNN accuracy as a function of k .

Inference: Accuracy is low for very small k (e.g., $k = 1$) because the model overfits to noise in individual training points, and it gradually improves and stabilizes as k increases into the range of roughly 5–15, after which accuracy plateaus and very slowly declines. This confirms the classic bias-variance trade-off in KNN: small k gives low bias but high variance (overfitting), while very large k increases bias by over-smoothing the decision boundary (underfitting). The empirically optimal region agrees closely with the $k = 5$ –6 chosen by GridSearchCV and RandomizedSearchCV.

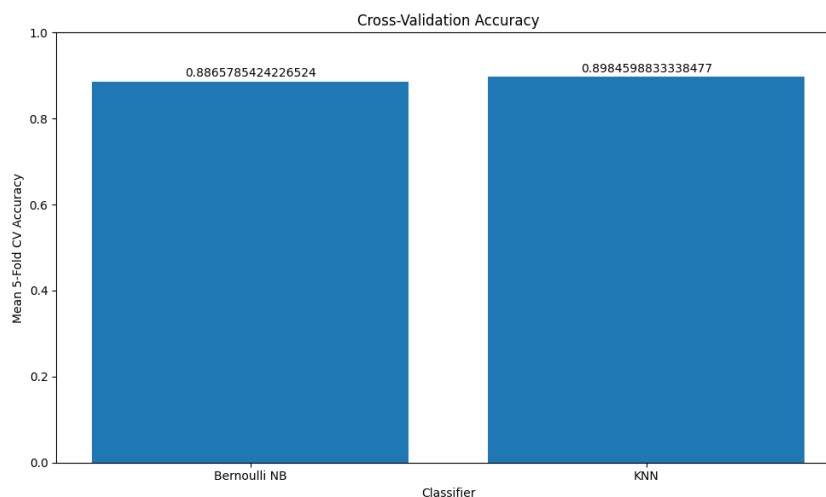


Figure 12: Mean 5-fold cross-validation accuracy: Bernoulli NB vs. KNN.

Inference: Under stratified 5-fold cross-validation, the default KNN model (0.8985 average accuracy) slightly outperforms Bernoulli NB (0.8866 average accuracy) on the training data. Both models show relatively low fold-to-fold variance, indicating stable generalization rather than a result driven by a lucky train-test split. This cross-validation view is more trustworthy than a single train-test split comparison, since it averages performance over five different data partitions.

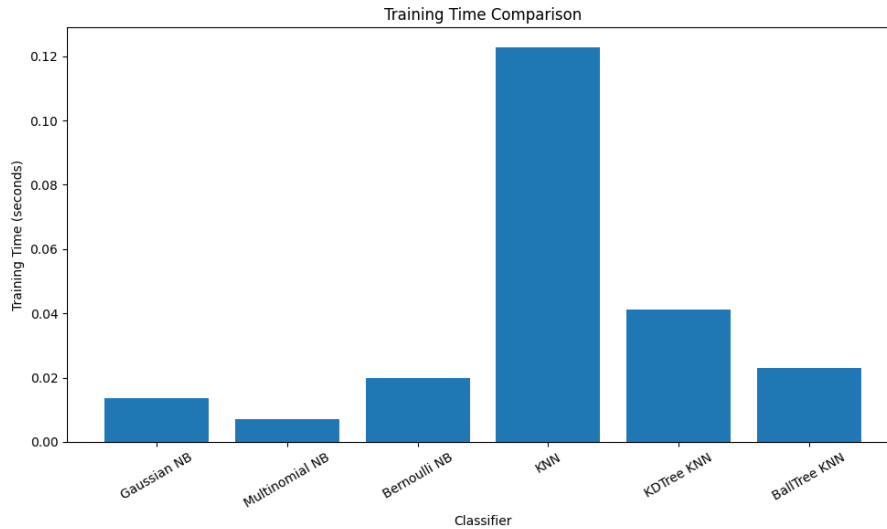


Figure 13: Training time comparison across classifiers.

Inference: As expected from the theoretical complexity of $O(nd)$, all three Naïve Bayes variants train almost instantly. KNN, KDTree KNN and BallTree KNN also show very low training times because “training” for KNN and its tree-based variants mainly consists of storing (or indexing) the training data rather than any iterative optimization – the KDTree/BallTree construction cost of $O(n \log n)$ is only noticeably larger than brute-force KNN’s near- $O(1)$ fit time at much larger dataset sizes than used here.

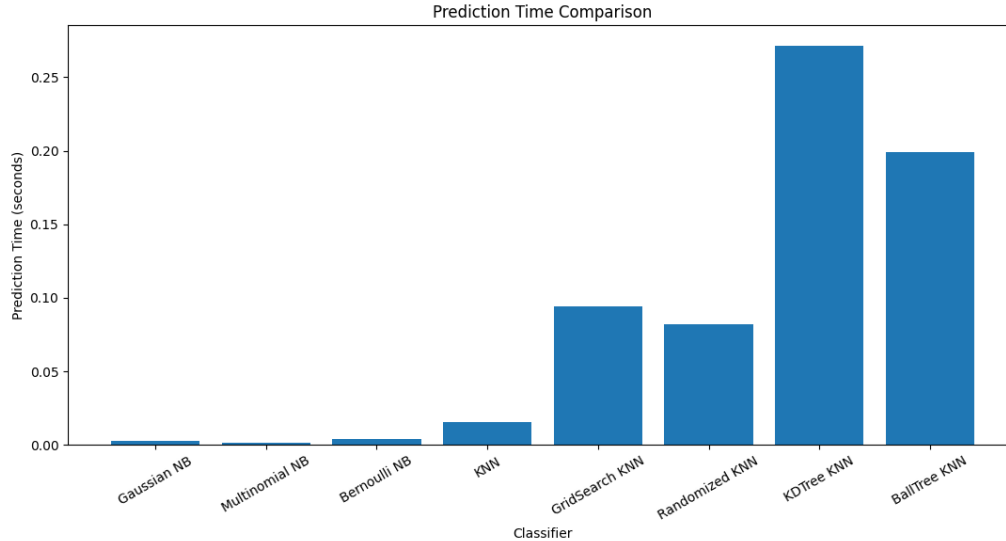


Figure 14: Prediction time comparison across classifiers.

Inference: Prediction time is where the algorithms differentiate most clearly: Naïve Bayes models predict fastest because their prediction cost is only $O(d)$ per sample, involving no comparison against the training set. KNN variants take noticeably longer at prediction time since they must compute distances to (a subset of) the training samples for every test point, consistent with the theoretical $O(nd)$ (brute-force) or $O(\log n)_{avg}$ (tree-based) prediction complexity. This is the classic trade-off of a lazy learner: near-zero training cost is paid for with higher prediction cost.

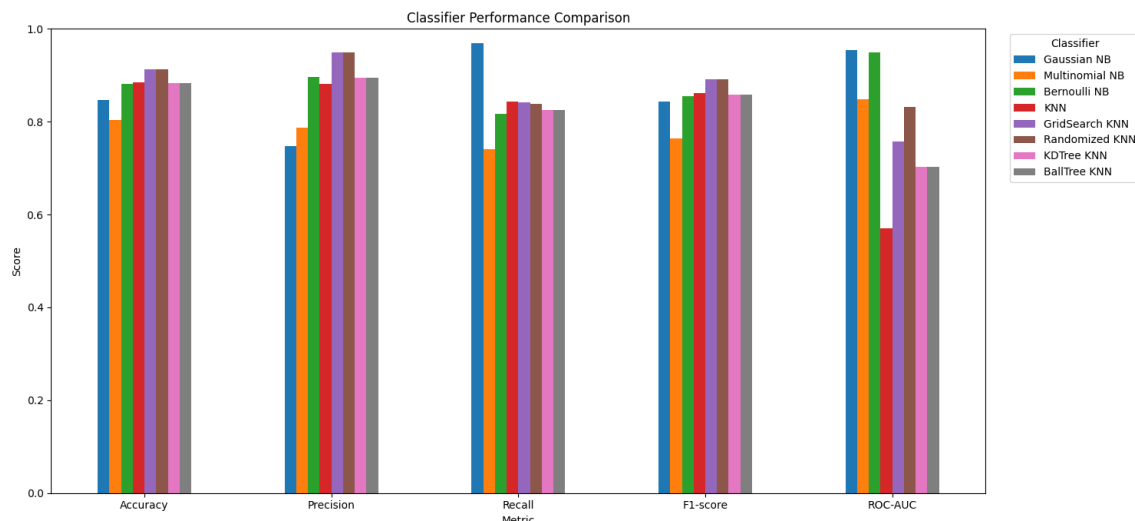


Figure 15: Side-by-side comparison of Accuracy, Precision, Recall, F1-score and ROC-AUC across all eight model variants.

Inference: This consolidated comparison shows that the GridSearchCV- and RandomizedSearchCV-tuned KNN models achieve the best overall accuracy and precision among all eight variants, while Gaussian NB retains the highest recall and ROC-AUC. Bernoulli NB offers the best balance among the untuned models. KDTree and BallTree KNN (both using default, unscaled-for-probability $k = 5$ settings) perform almost identically to each other, as expected since they compute the exact same nearest neighbors – they differ only in the search strategy used to find them, not in the result.

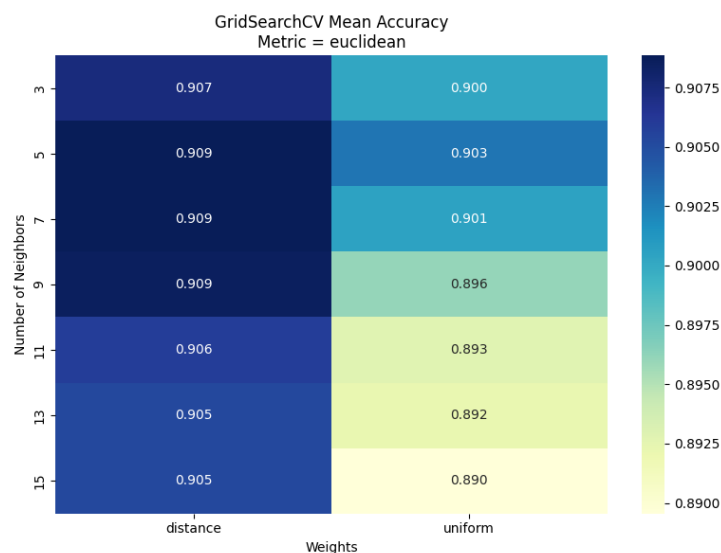


Figure 16: GridSearchCV mean CV accuracy heatmap (Euclidean metric).

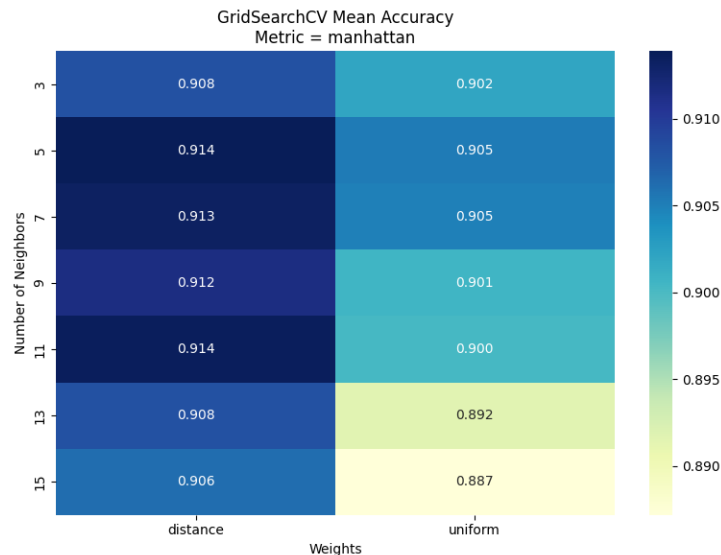


Figure 17: GridSearchCV mean CV accuracy heatmap (Manhattan metric).

Inference: Across both distance metrics, “distance”-weighted voting consistently outperforms “uniform” voting, and the Manhattan metric gives a slightly higher peak accuracy (about 0.914 at $k = 5$) than Euclidean (about 0.909 at $k = 5$ –9). Accuracy is fairly stable for k between 3 and 9 and gradually decreases for larger k under uniform weighting, while distance weighting is more robust to the choice of k . This is why GridSearchCV ultimately selected `metric='manhattan'`, `weights='distance'`, `n_neighbors=5` as the best configuration.

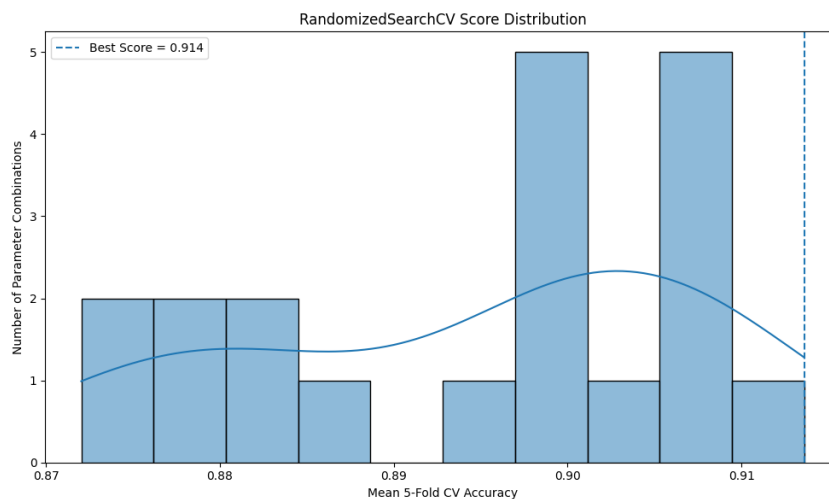


Figure 18: Distribution of mean CV accuracy across the 20 RandomizedSearchCV parameter combinations sampled.

Inference: Most of the randomly sampled hyperparameter combinations cluster in the 0.85–0.91 accuracy range, with a small number of combinations reaching the best observed score of about 0.9136 (marked by the dashed line). This shows that RandomizedSearchCV, despite evaluating far fewer combinations (20) than the exhaustive GridSearchCV ($7 \times 2 \times 2 = 28$), was still able to locate a near-optimal region of the hyperparameter space, at a lower computational cost.

11 Discussion

Across all experiments, the tuned KNN models (via GridSearchCV and RandomizedSearchCV) achieved the highest accuracy and precision, while Gaussian Naïve Bayes achieved the highest recall and ROC-AUC, making it attractive when missing an actual spam email (a false negative) is considered more costly than misclassifying a legitimate email. Bernoulli NB offered the best balance among the Naïve Bayes variants and matched or exceeded default KNN performance without any hyperparameter search, and it is by far the fastest to train and predict.

The comparison between GridSearchCV and RandomizedSearchCV showed that both search strategies converged to very similar hyperparameters (`manhattan` distance, `distance` weighting, $k \approx 5-6$) and near-identical cross-validation accuracy, confirming that RandomizedSearchCV is a computationally cheaper but still reliable alternative to an exhaustive grid search on this dataset. KDTree and BallTree produced identical predictive performance to each other, as they are two different but mathematically equivalent strategies for finding exact nearest neighbors; their difference lies purely in indexing/search efficiency, which only becomes significant on much larger or higher-dimensional datasets.

In terms of theoretical vs. practical complexity, the measured training and prediction times were consistent with the theoretical complexity table: Naïve Bayes trains and predicts fastest ($O(nd)$ / $O(d)$), while KNN-based methods have near-zero training cost but noticeably higher prediction cost, matching the expected $O(nd)$ (brute-force) or $O(\log n)_{avg}$ (tree-based) prediction complexity.

Strengths: The experiment covers a broad set of models, hyperparameter tuning strategies and evaluation metrics, giving a well-rounded comparison. Stratified cross-validation was used to guard against biased train-test splits given the class imbalance.

Limitations: The default (untuned) KNN model's low ROC-AUC illustrates that `predict_proba` from a majority-vote KNN can be a poor probability estimator; this is a known limitation rather than a modeling error and future work could compare against distance-weighted probability estimates directly. Multinomial NB is not perfectly suited to the (largely continuous, non-count) Spambase features, which likely also handicaps its reported performance relative to Gaussian/Bernoulli NB.

Possible improvements: Feature selection or dimensionality reduction (e.g., removing highly correlated features) could reduce KNN's prediction time without a large accuracy loss; testing additional distance metrics (e.g., Chebyshev) and ensembling the best Naïve Bayes and KNN models could further improve recall and precision simultaneously.

12 Conclusion

This experiment successfully built and compared Gaussian, Multinomial and Bernoulli Naïve Bayes classifiers alongside multiple KNN configurations (default, GridSearchCV-tuned, RandomizedSearchCV-tuned, KDTree-based and BallTree-based) for spam email classification on the Spambase dataset. GridSearchCV-tuned KNN (`manhattan`, `distance`-weighted, $k = 5$) achieved the best overall accuracy (0.9133) and precision (0.9497), while Gaussian NB achieved the best recall (0.9694) and ROC-AUC (0.9550). Bernoulli NB offered the most favorable balance of speed and accuracy among the Naïve Bayes variants. The measured training/prediction times matched the expected theoretical time-complexity ordering of the algorithms, confirming that Naïve Bayes is preferable for very

large datasets or latency-sensitive applications, while a well-tuned KNN is preferable when maximizing accuracy/precision is the priority and the dataset size is manageable.

13 References

1. Scikit-learn Documentation, <https://scikit-learn.org/stable/>.
2. A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*, 2nd ed., O'Reilly Media, 2019.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. Somesh24, "Spambase Dataset," Kaggle. [Online]. Available: <https://www.kaggle.com/datasets/somesh24/spambase>.